

1. Introducción

La serialización es la conversión de un objeto a una secuencia (stream) de bytes.

Este stream es el que se transfiere por la red o se almacena en la base de datos o se graba en un fichero.

2. Serialization y Deserialization

La serialización es instance-independent, esto es, podemos serializar en una plataforma y deserializar en otra.

Para llevar a cabo la serialización de una clase es necesario que ésta implemente la interface Serializable que contiene entre otros estos dos métodos.

```
private void writeObject(java.io.ObjectOutputStream out)
    throws IOException

private void readObject(java.io.ObjectInputStream in)
    throws IOException, ClassNotFoundException;
```

Ejemplo:

```
public class Person implements Serializable {
    private static final long serialVersionUID = 1L;
    static String country = "ITALY";
    private int age;
    private String name;
    transient int height;

    // getters and setters
}
```

A tener en cuenta:

Se recomienda que la clase contenga el atributo `private static final long serialVersionUID = 1L;`. Si no lo hacemos nosotros lo crea el IDE y es dependiente del compilador, lo que puede darnos problemas (*InvalidClassExceptions*.)

- Los miembros static no se serializan
- Se emplea *transient* para indicar que un atributo no se serializa. Esto significa ese atributo del objeto que no se escribirá en el fichero (en caso de que necesitemos serializar para gestionar el contenido de ficheros) o no se enviará por red

Los motivos por los que querríamos que un atributo fuera transient son (entre otros):

- Seguridad. No queremos que esa información salga del ordenador, por ejemplo.
- Optimización. No necesitamos enviar esa información

En el ejemplo siguiente se describe un test en el que se serializa la clase Persona guardando en un fichero el stream.

```
@Test
public void whenSerializingAndDeserializing_ThenObjectIsTheSame() ()
    throws IOException, ClassNotFoundException {
    Person person = new Person();
    person.setAge(20);
    person.setName("Joe");

    FileOutputStream fileOutputStream
        = new FileOutputStream("yourfile.txt");
    ObjectOutputStream objectOutputStream
        = new ObjectOutputStream(fileOutputStream);
    objectOutputStream.writeObject(person);
    objectOutputStream.flush();
    objectOutputStream.close();

    FileInputStream fileInputStream
        = new FileInputStream("yourfile.txt");
    ObjectInputStream objectInputStream
        = new ObjectInputStream(fileInputStream);
    Person p2 = (Person) objectInputStream.readObject();
    objectInputStream.close();
}
```

```
    assertTrue(p2.getAge() == person.getAge());
    assertTrue(p2.getName().equals(person.getName()));
}
```

3. Más...sobre Serialization

- Cuando una clase implementa la interface *java.io.Serializable* todas sus clases hija son también serializables.
- Cuando una clase implementa la interface *java.io.Serializable* todos sus atributos que sean objeto (dicho de otra forma los objeto referenciados por otros objetos) son también serializables.

Ejemplo:

```
public class Person implements Serializable {
    private int age;
    private String name;
    private Address country; // must be serializable too
}
```

4. Custom Serialization

Se puede puentear (override) el comportamiento por defecto modificando los métodos por defecto.

¿Cuándo? Por ejemplo para serializar un objeto que tiene atributos no serializables

¿Por qué?

- Porque queremos llevar a cabo la serialización de ese transient una forma diferente a la automática

- Porque el atributo transient no cambia, y para optimizar no lo serializo al hacer writeObject pero lo necesito restituir al hacer readObject

¿Cómo?

Respetando estas signatures:

```
private void writeObject(ObjectOutputStream oos)
    throws IOException {
}

private void readObject(ObjectInputStream ois)
    throws ClassNotFoundException, IOException {
}
```

Ejemplo:

```
public class Employee implements Serializable {
    private static final long serialVersionUID = 1L;
    private transient Address address;
    private Person person;

    // setters and getters

    private void writeObject(ObjectOutputStream oos)
        throws IOException {
        oos.defaultWriteObject();
        // aquí podríamos indicar cómo serializar el transient
        oos.writeObject(address.getHouseNumber());
    }

    private void readObject(ObjectInputStream ois)
        throws ClassNotFoundException, IOException {
        ois.defaultReadObject();
        Integer houseNumber = (Integer) ois.readObject();
        Address address = new Address();
        address.setHouseNumber(houseNumber);
    }
}
```

```
public class Address {
    private int houseNumber;
```

```
// setters and getters  
}
```

¿Por qué no es necesario indicar `@Override` delante de los métodos `writeObject` y `readObject`?

Porque no se trata de una interfaz pública de Java

Recordatorio: `@Override` es una anotación NO obligatoria. Cuando se incluye es una forma de advertirle al compilador que se asegure de que el método que estamos sobrescribiendo esté bien definido (nombre, parámetros y valor de retorno).

5. Resumen

Por tanto:

- Podemos enviar y recibir objetos a través de la red utilizando las clases ***ObjectInputStream*** y ***ObjectOutputStream*** siempre y cuando estos objetos implementan la interface ***Serializable***.
- La clase ***String*** implementa la interfaz ***Serializable*** y por eso también se puede enviar de esta forma.
- Se emplea ***transient*** para indicar que un atributo **no viaja** por la red.
- **No** hay que **implementar ningún método** de la Interface ***Serializable*** siempre y cuando **no** queramos **modificar** el **comportamiento** por defecto del ***transient***.
- Hay que tener en cuenta que si Servidor y Cliente van a utilizar una clase definida por el usuario, esta **clase debe encontrarse en ambos (la clase tiene que tener el mismo nombre y mismo paquete en ambos host)**. De lo contrario, el método `readObject()` lanzará la excepción ***ClassNotFoundException***.